

LLMs Part 2: Transformers

Ronald A. Sumida

rsumida@innovationinsoftware.com

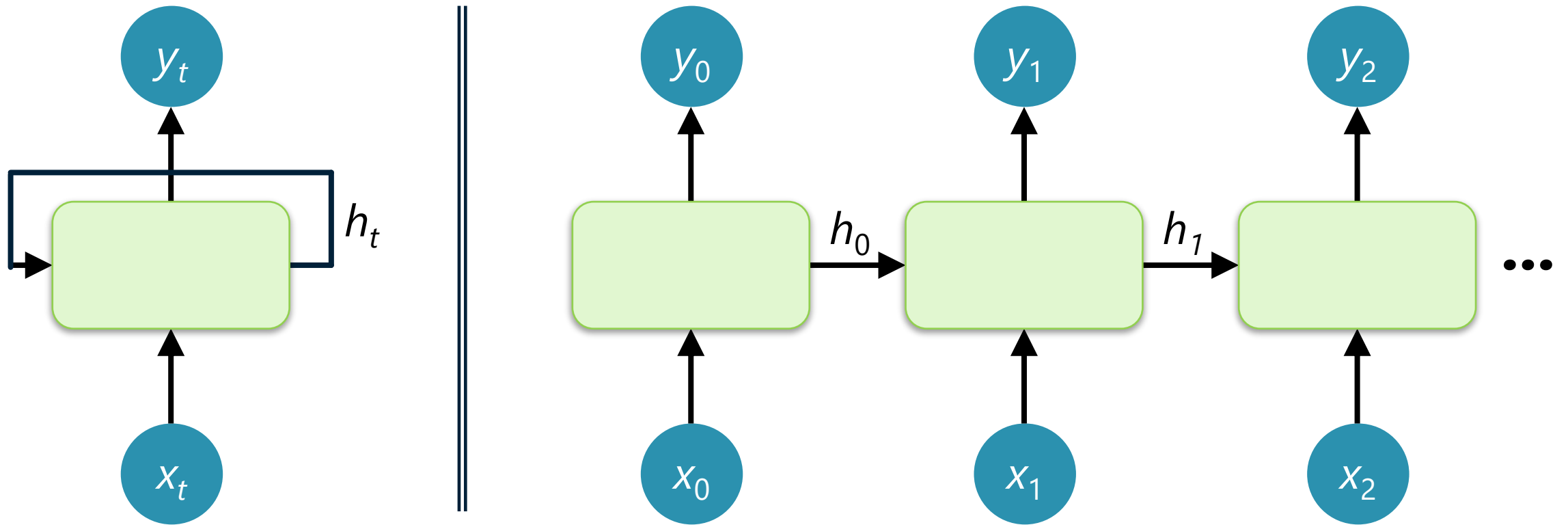


Outline

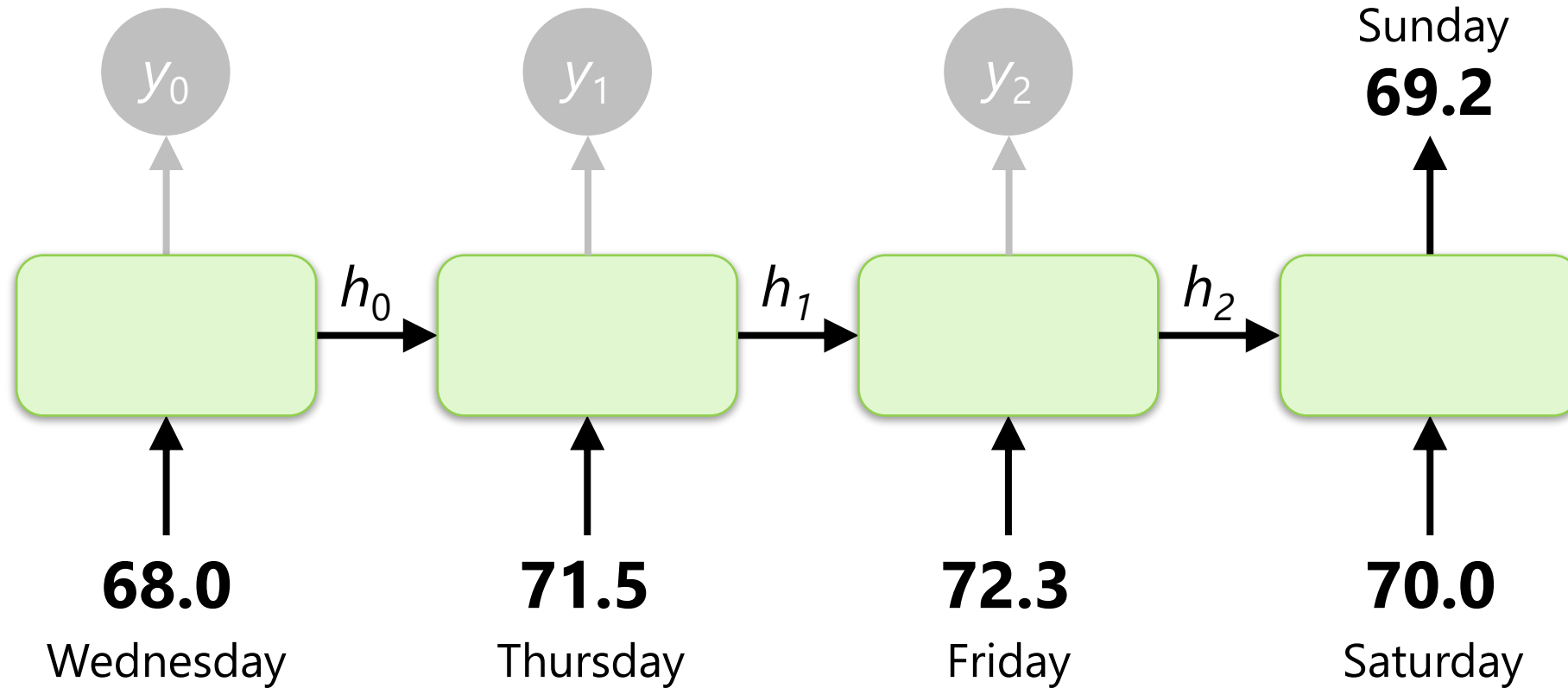
- Recurrent Neural Networks (RNN)
- Self-attention
- Transformers

Recurrent Neural Networks (RNNs)

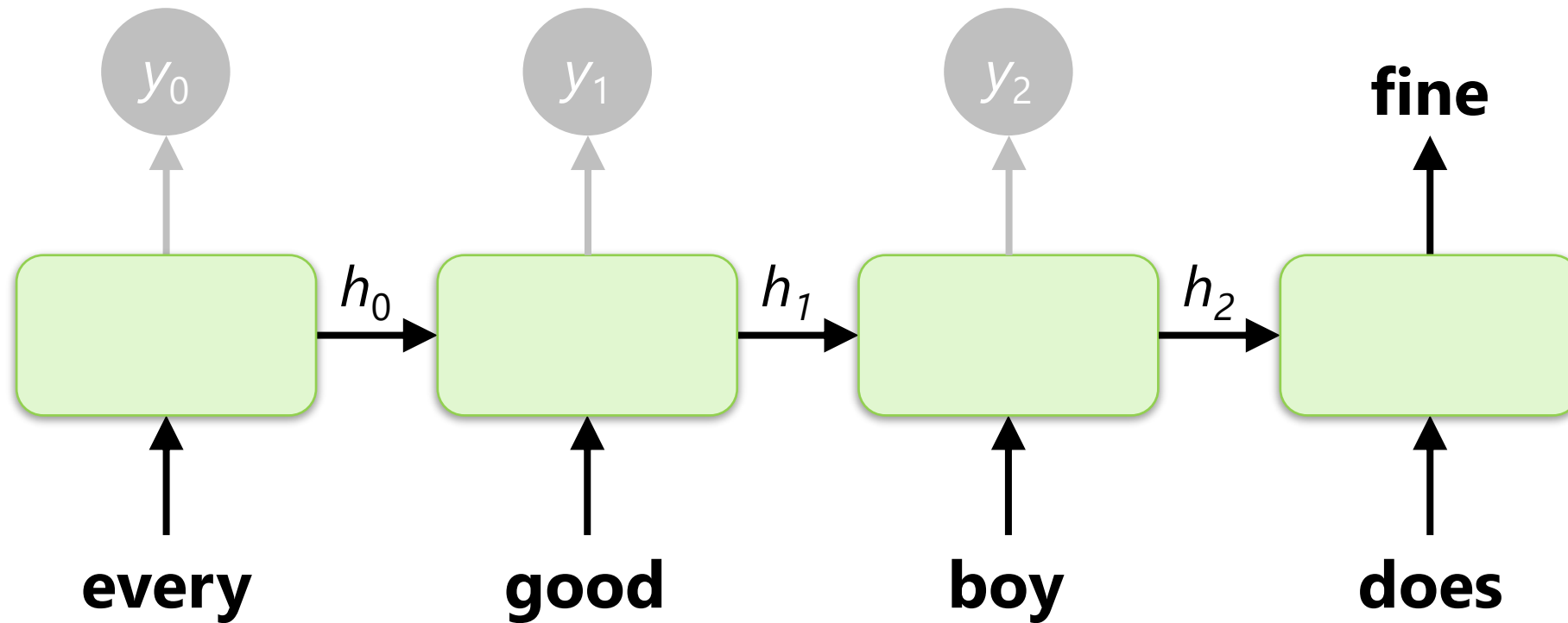
- Neural networks designed to process sequential data



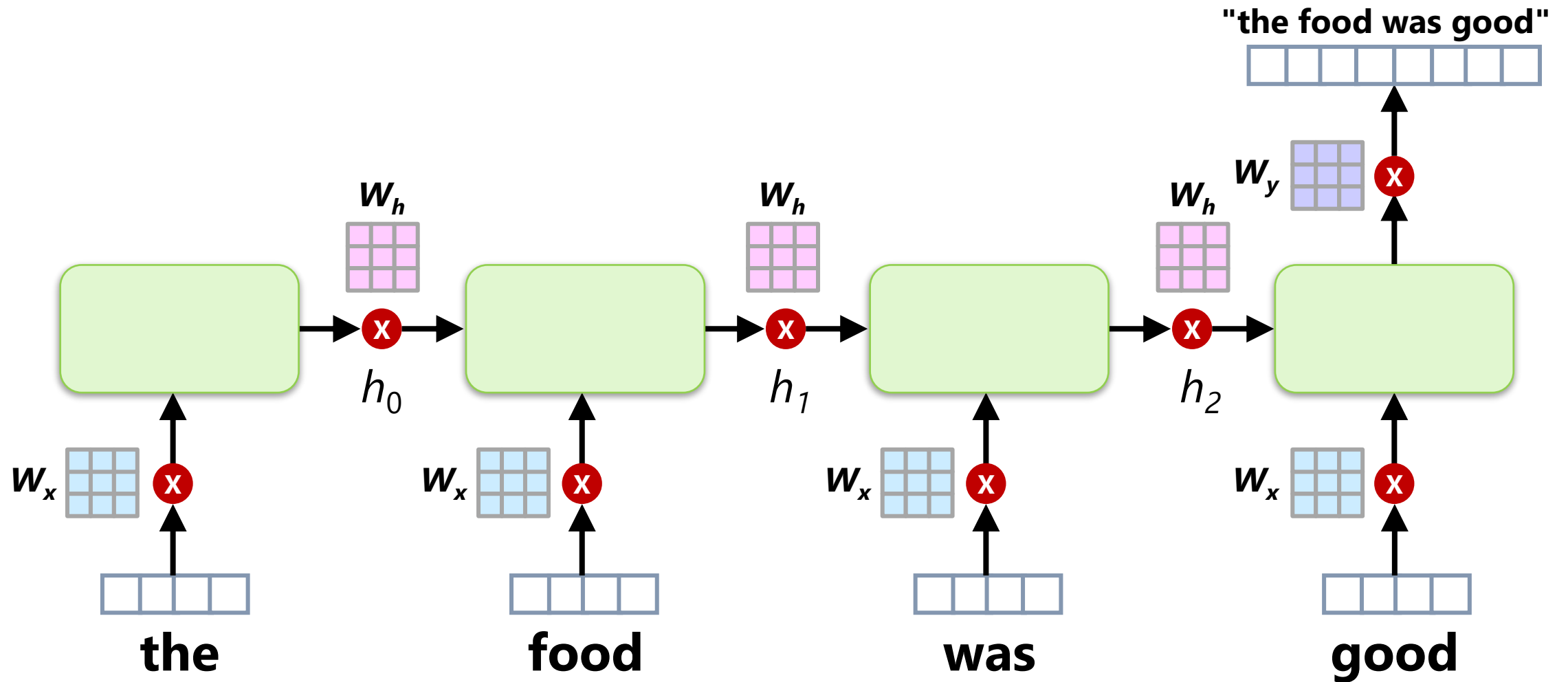
Predicting Tomorrow's High Temperature



Predicting the Next Word in a Series

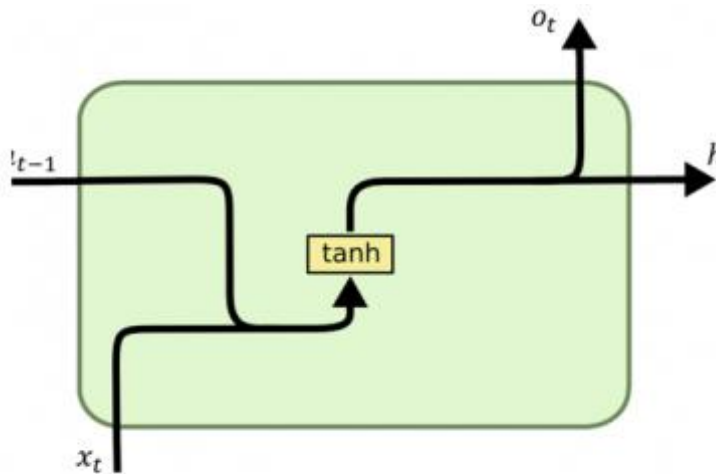


Classifying Text



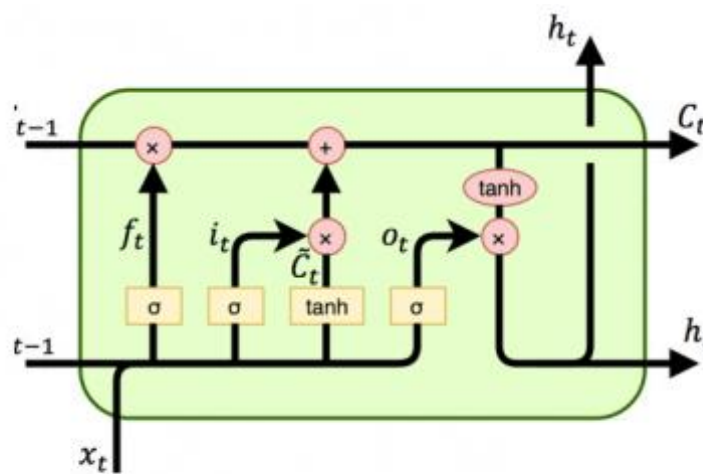
Recurrent Layers in Keras

SimpleRNN



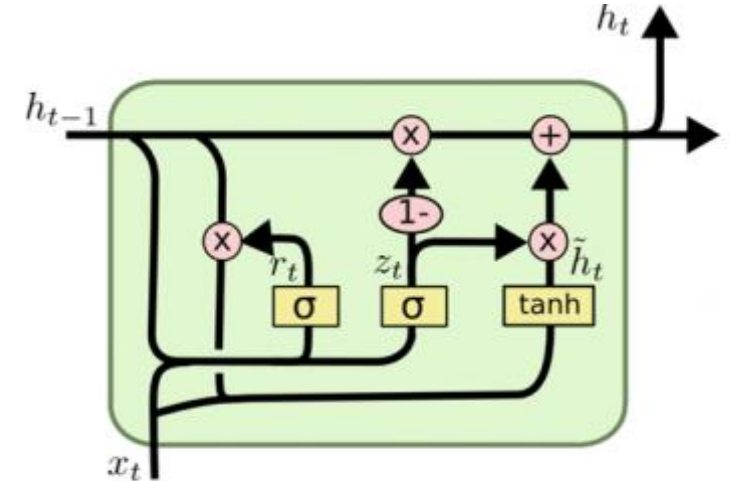
Combines **input from previous cell** with **output from previous cell** and uses **tanh activation** to normalize the result

LSTM



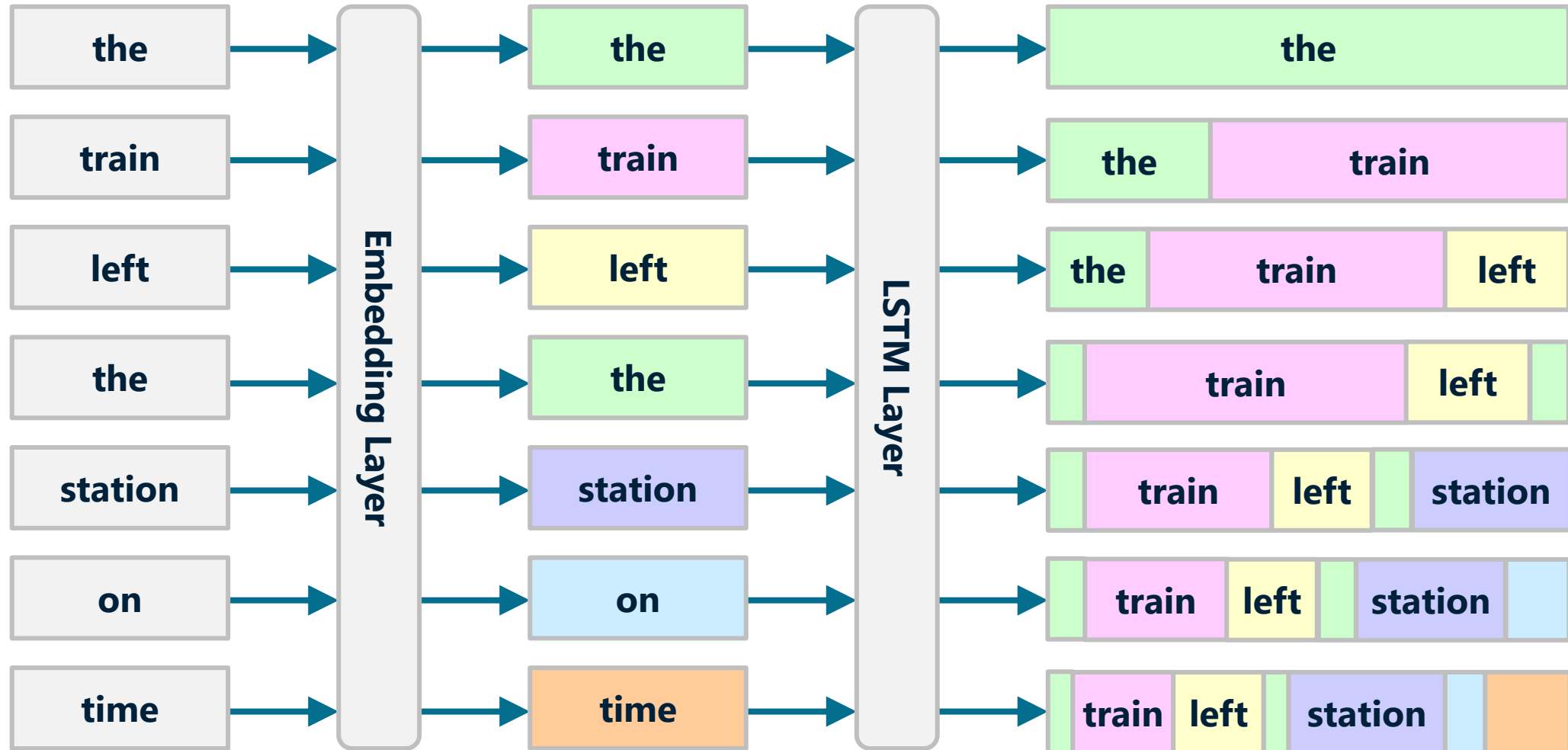
Uses trainable **forget gate**, **input gate**, and **output gate** to discard less relevant information and add weight to more relevant information

GRU

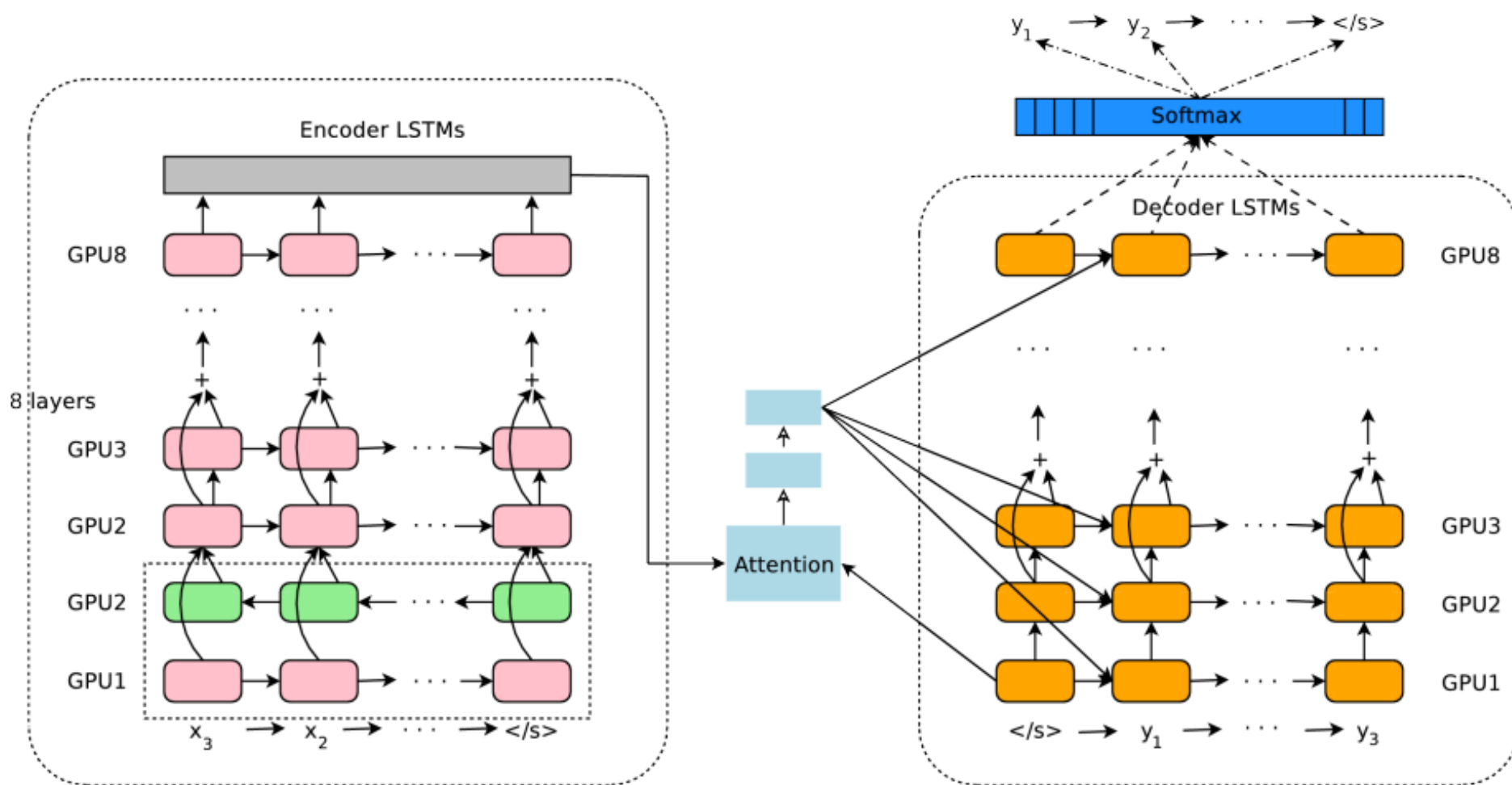


Uses **reset gate** and **update gate** to accomplish the same goal with fewer gates

Processing Text Sequences Recurrently



Neural Machine Translation (NMT)



"Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation" (<https://arxiv.org/abs/1609.08144>)

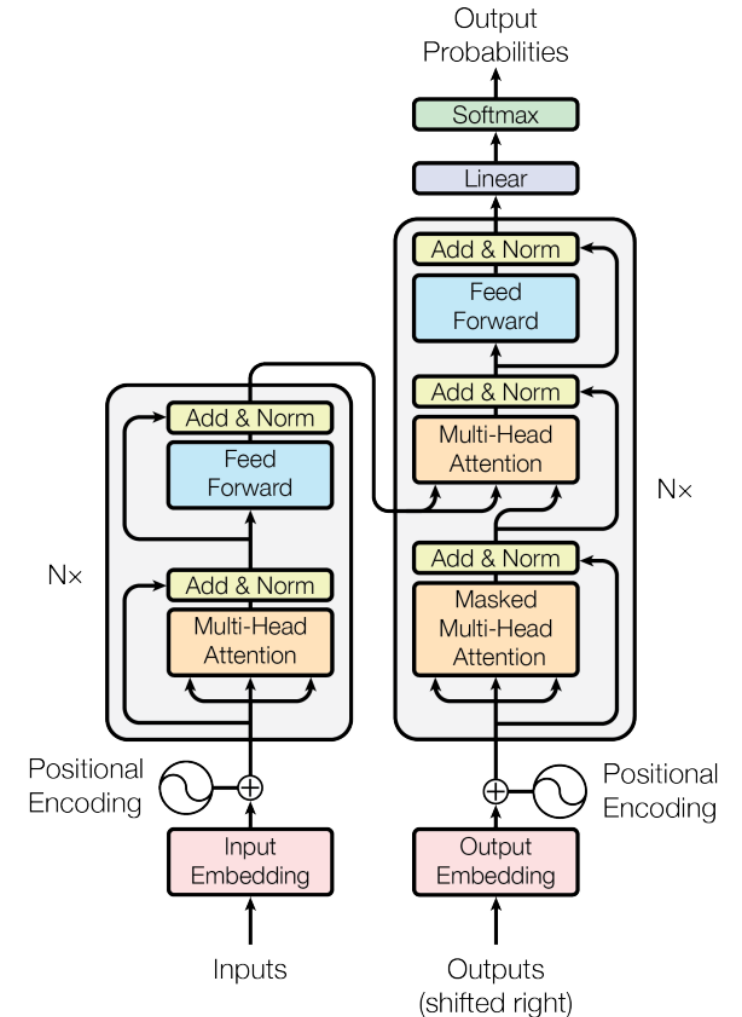
Demo

Sentiment Analysis with RNNs

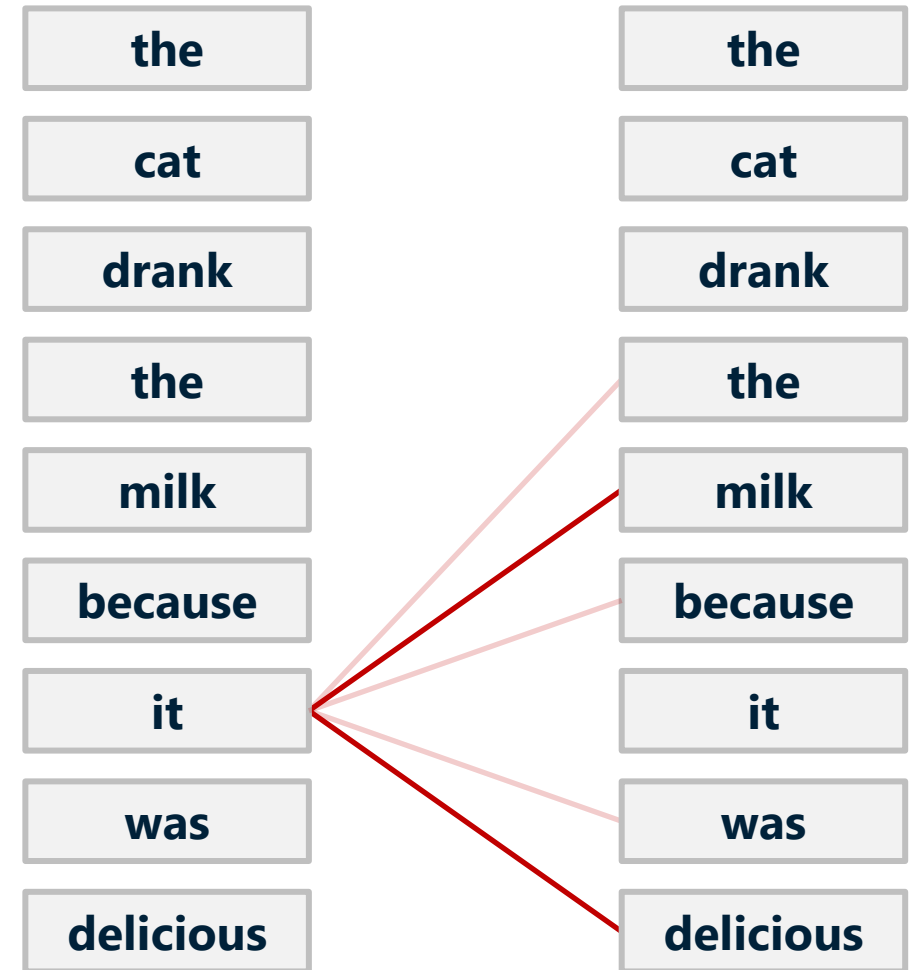
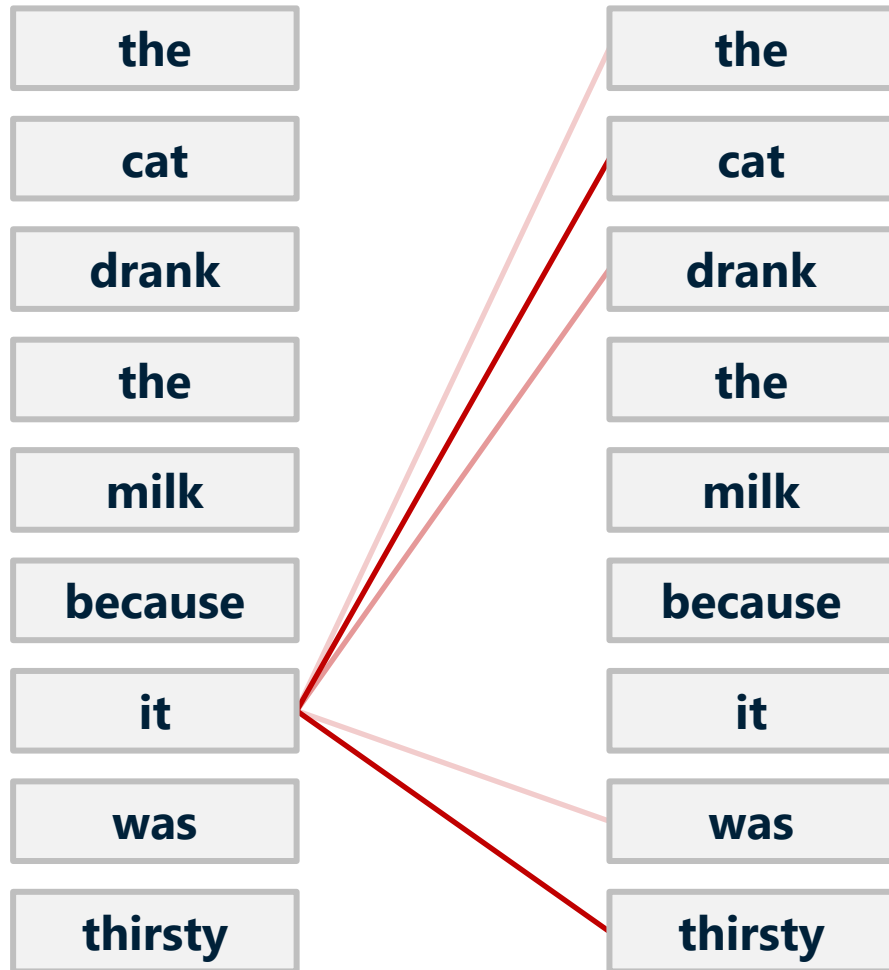


Transformers

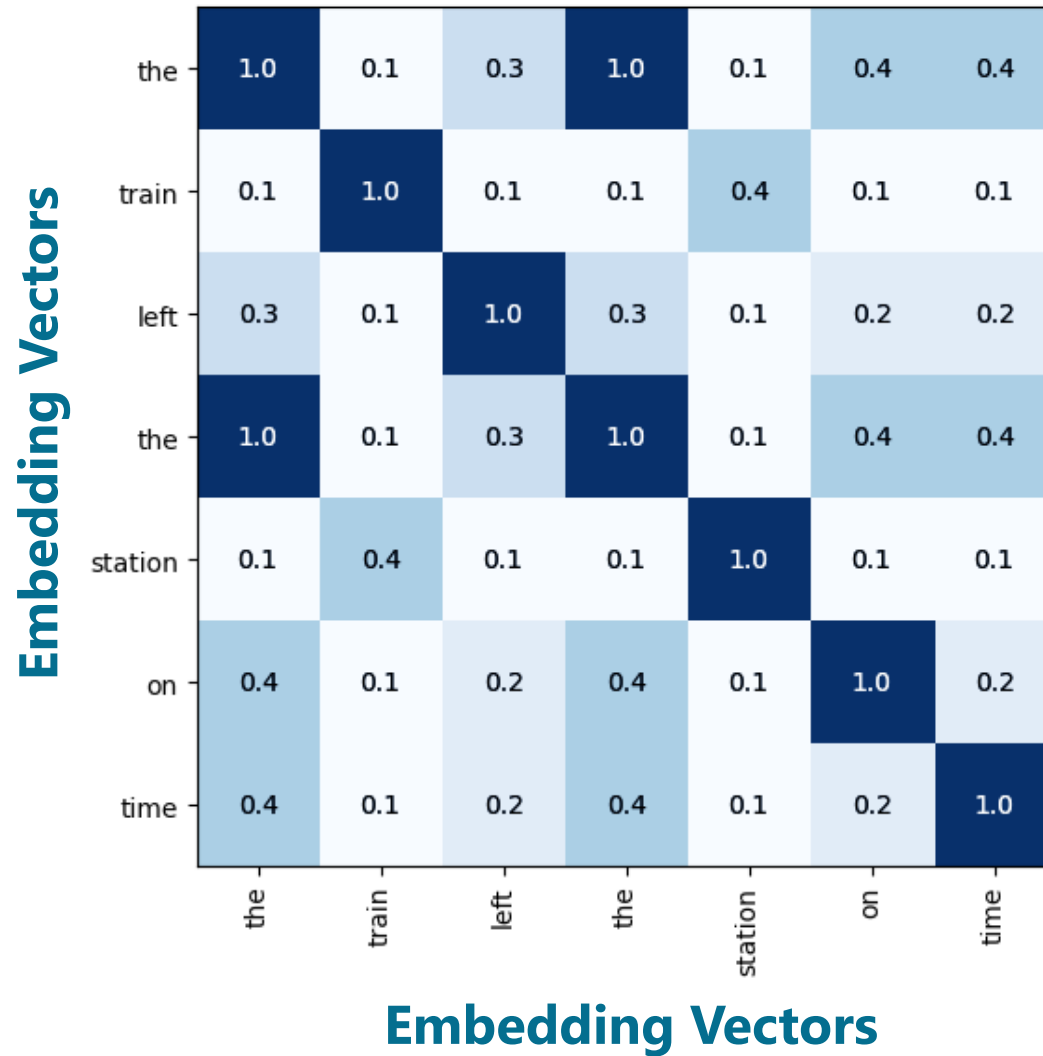
- Introduced in landmark 2017 paper entitled **Attention is All You Need**
- Use **self-attention** to turn static word/token embeddings into context-aware embeddings
 - Discern between different meanings of the same word (polysemy), connect pronouns to subjects, connect adjectives to nouns, and more
- Attention heads train in parallel and connect words independent of distance



Self-Attention



Dot Products of Word Embeddings



Context aware embeddings

- Goal: Generate *attention scores*
 - Quantify how embeddings should be blended together
- Dot products of each pair of embedding vectors is not sufficient
 - Only quantifies the syntactic similarity of the words
- Need to generate *context aware embeddings*
 - Based on how much one word attends to the other in a sentence

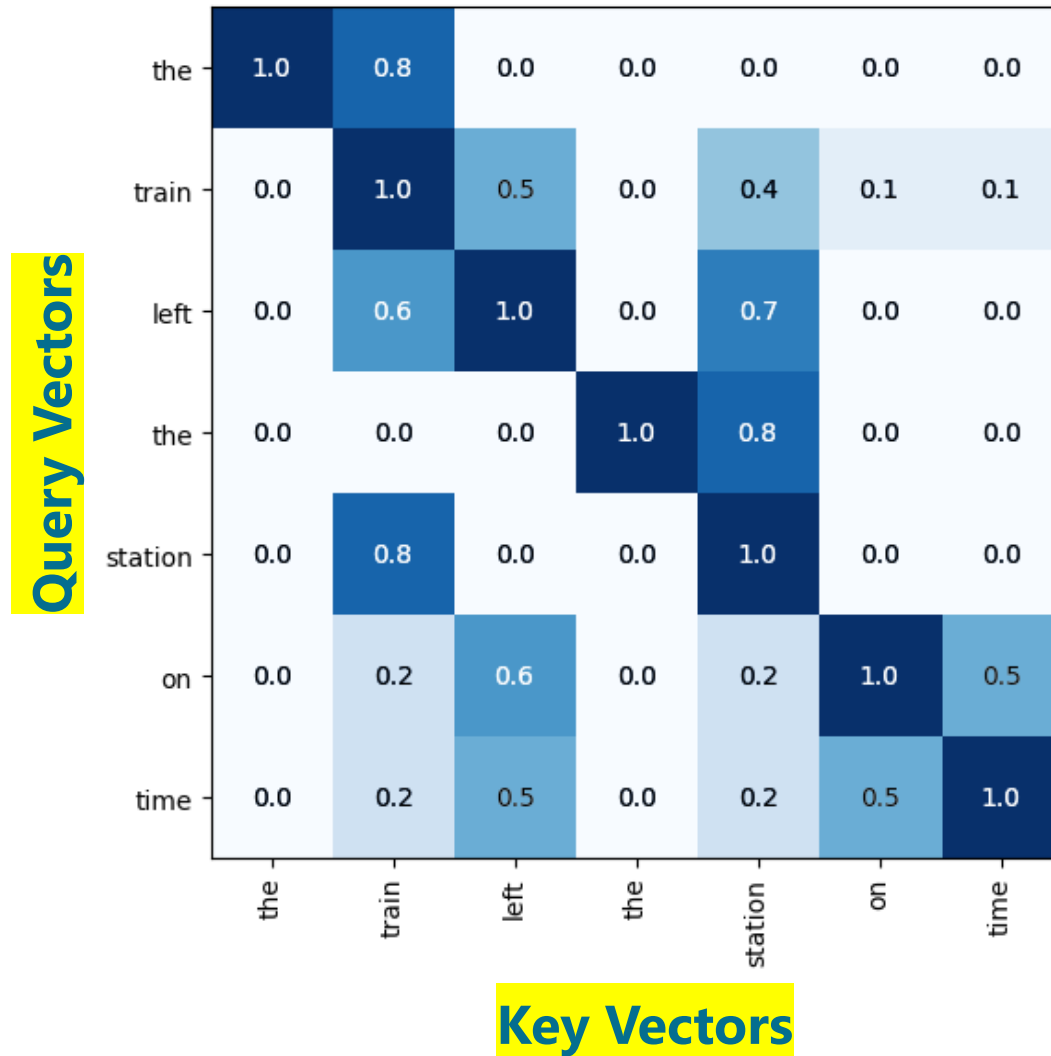
Core attention mechanism concepts

- Query
 - What you're looking for
- Key
 - What each word (or token) offers
- Value
 - Contains actual info to be passed along

Query, Key and Value example

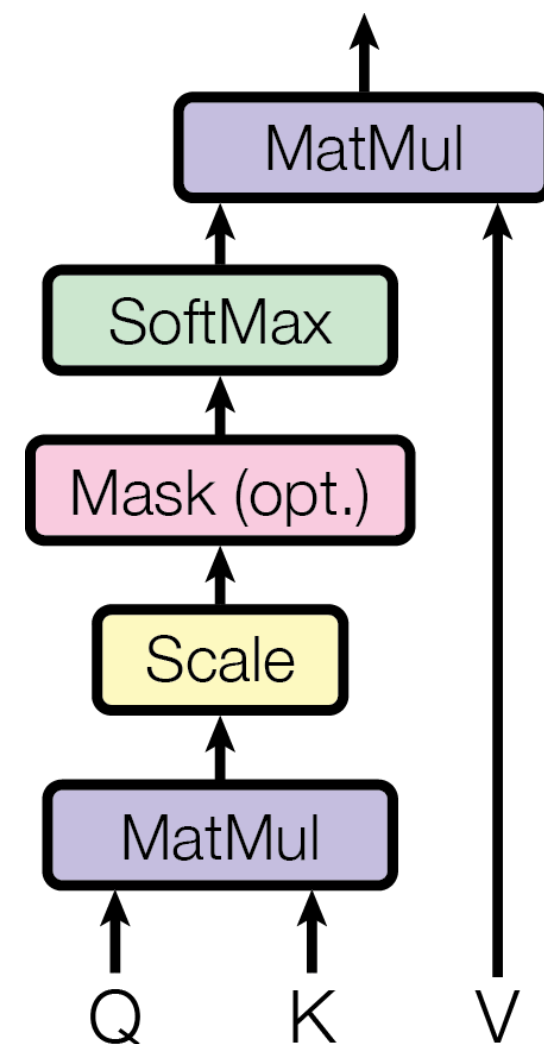
- Understanding the word “bank” in a sentence
 - Ground bordering river vs. financial institution
- Query
 - What does bank mean here?
- Key
 - Represent surrounding words
 - E.g., “river” or “money”
- Value
 - Decide which meaning (value) deserves attention

Attention Scores

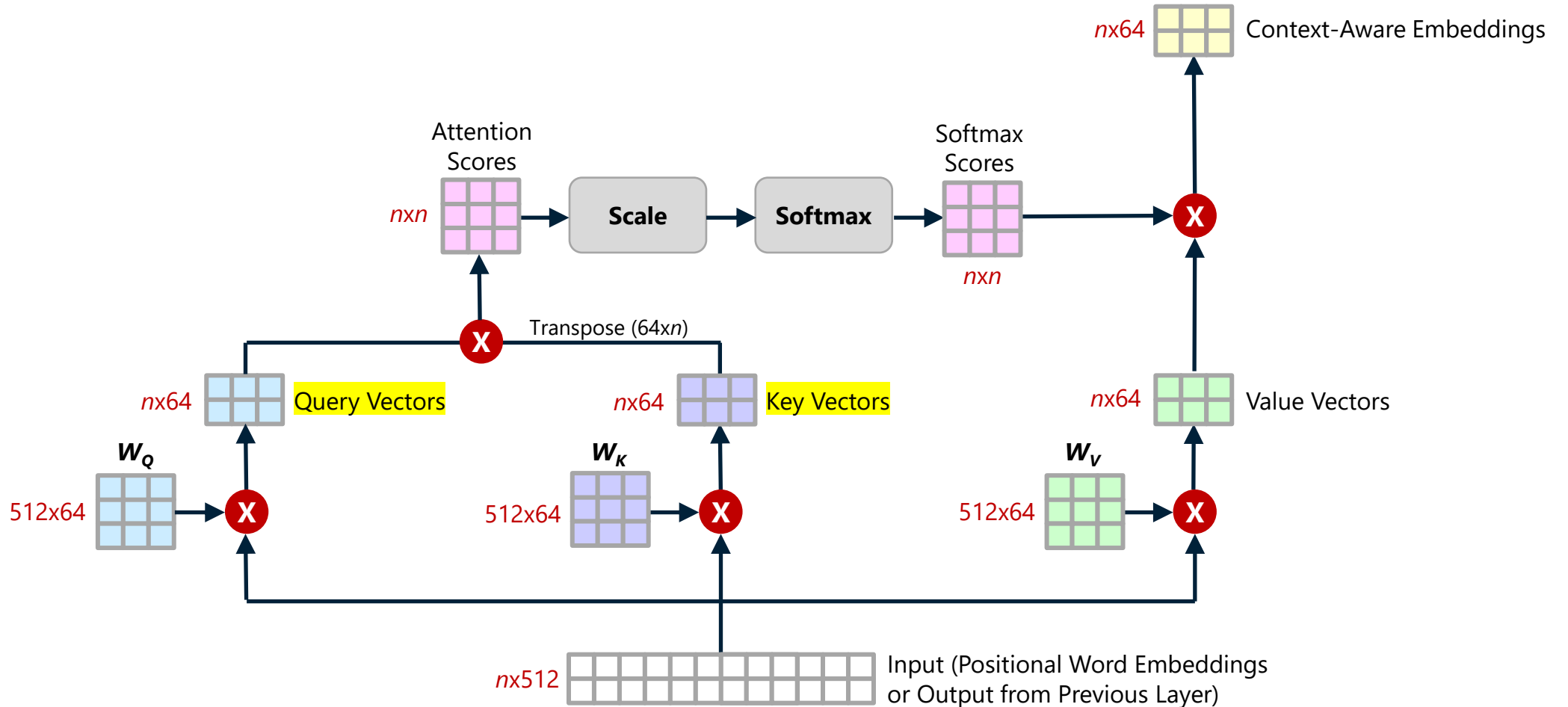


Scaled Dot-Product Attention

- Relies on three learned weight matrices
 - Query matrix (W_Q)
 - Key matrix (W_K)
 - Value matrix (W_V)
- Multiply input embeddings by W_Q , W_K , and W_V to produce **query**, **key**, and **value** vectors (Q , K , and V)
- Compute dot product of each **query** vector and **key** vector to generate attention scores
- Scale attention scores, apply softmax, and multiply by **value** vectors to produce output

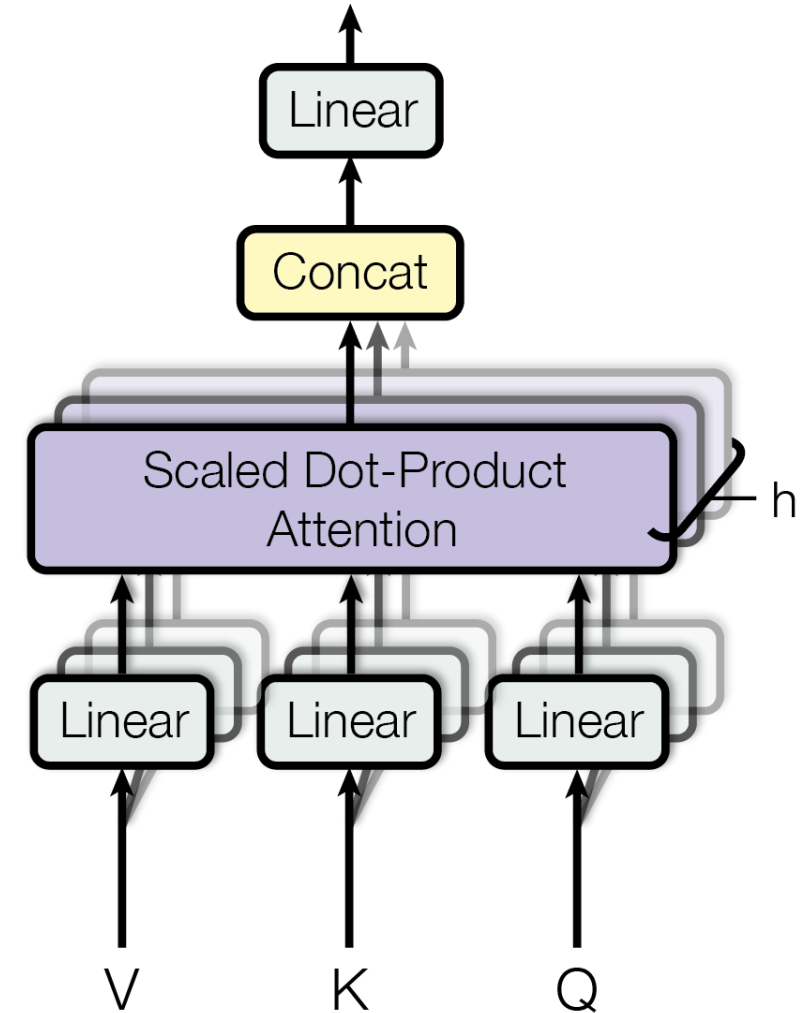


How Scaled Dot-Product Attention Works

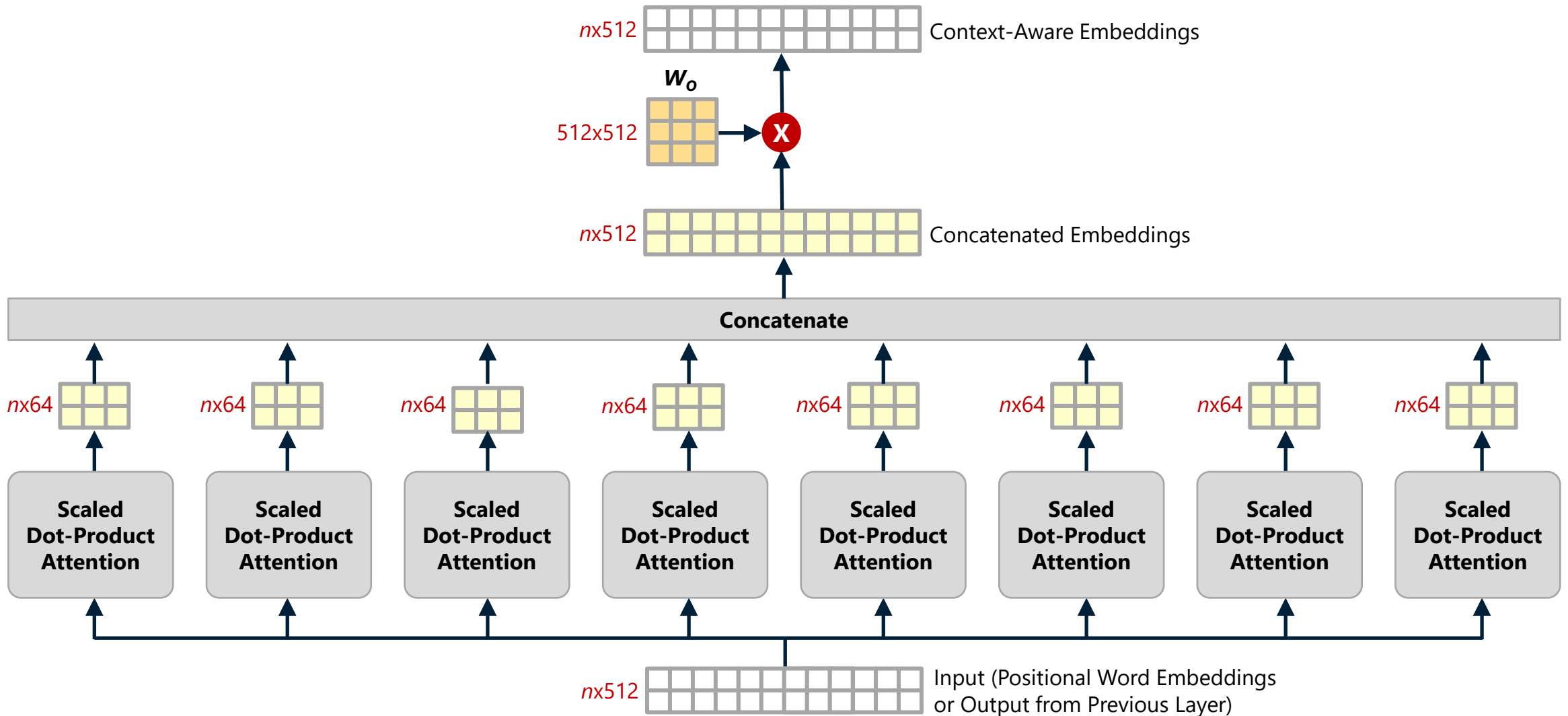


Multi-Head Attention

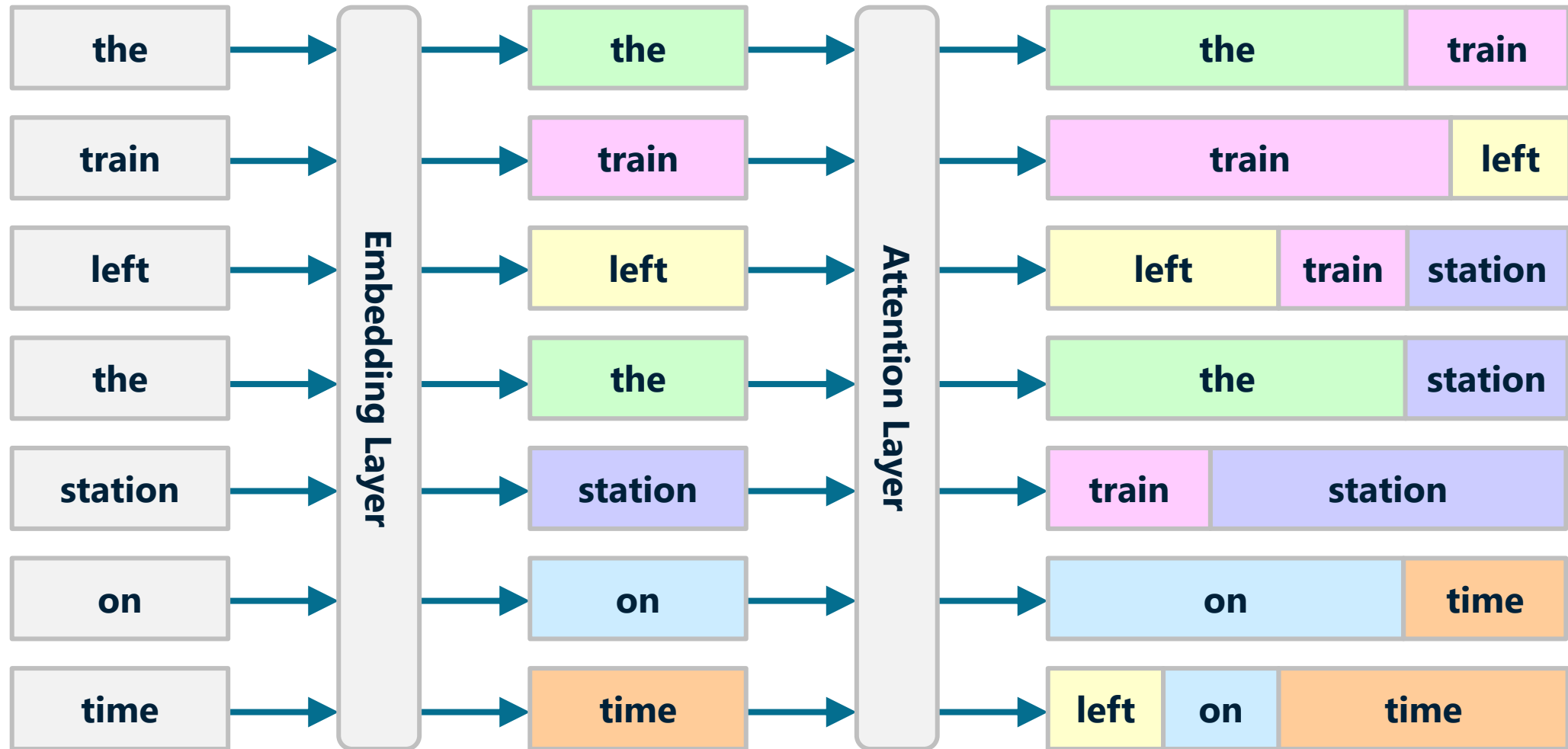
- Each layer has multiple attention heads, each with its own W_Q , W_K , and W_V
 - Each initialized with random numbers
- Concatenate output from each head, and multiply with another learned matrix (W_O) to produce final output
 - Output from head n serves as input to $n+1$
- Enables different attention scores to be computed for the same input sequence



How Multi-Head Attention Works

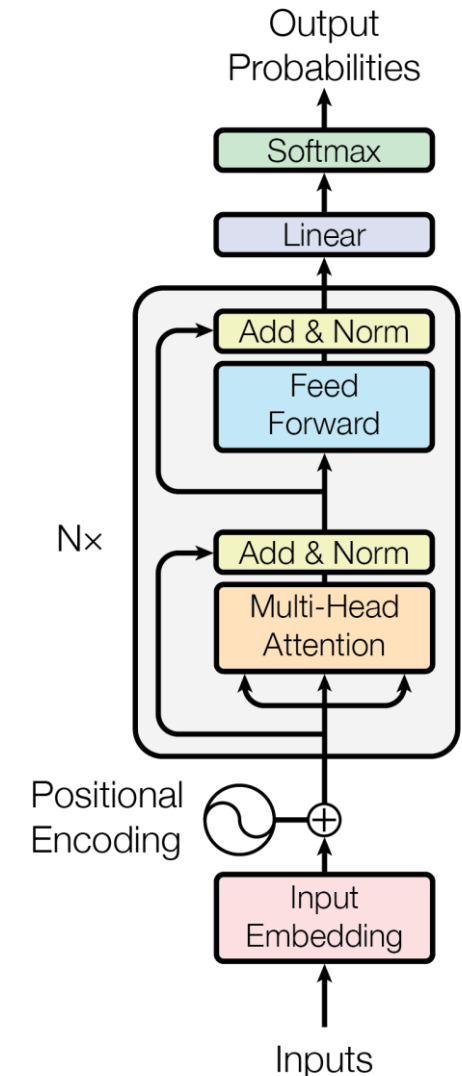


Context-Aware Word Embeddings



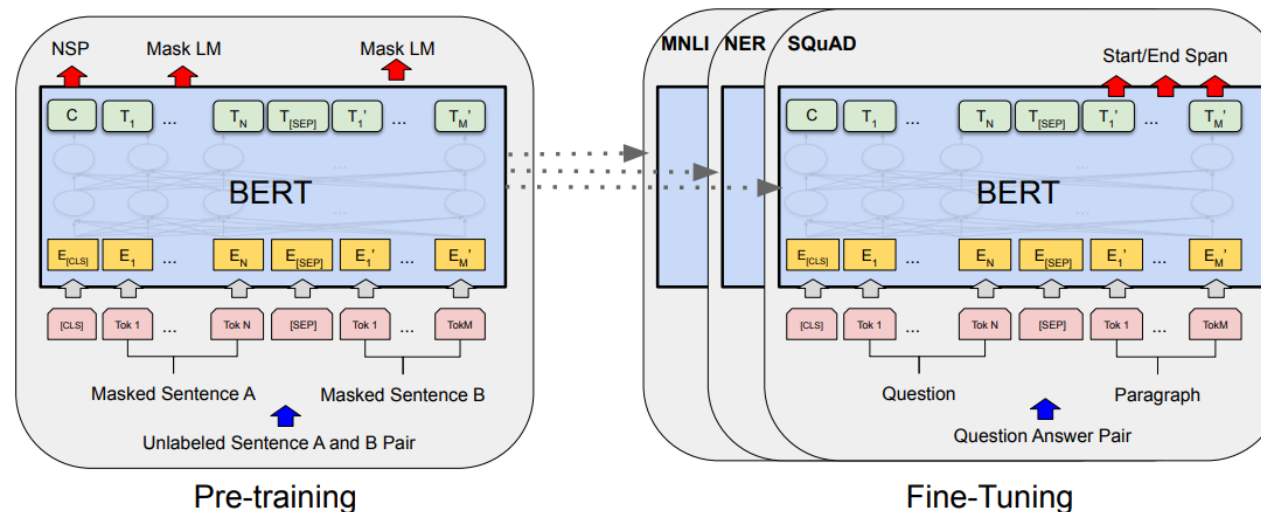
Encoder-Only Models

- Produce highly context-aware vectors representing entire input sequences
- Excel at **text-classification tasks** such as sentiment analysis and spam filtering
 - Also used for named entity recognition and extractive question answering
- Pretrained models such as BERT and its many derivatives can be fine-tuned for specific NLP tasks to achieve higher accuracy than models trained from scratch



BERT

- Bidirectional Encoder Representations from Transformers
- Encoder-only model built by Google and instilled with language understanding by pretraining with billions of words
- Can be fine-tuned to perform a variety of NLP tasks



Masked Language Modeling (MLM)

- Turns unlabeled text into training ground for language structure
- During pretraining, a specified percentage (usually 15%) of the tokens are randomly masked (dropped) from the training sequences, and the model is trained to predict the missing words

?	and	seven	years	?	our
fathers	?	forth	on	this	continent
a	new	?	conceived	in	liberty
and	dedicated	?	the	proposition	?

Fine-Tuning a BERT Model

```
# Load a pretrained BERT transformer and set num_labels=2 for binary classification
model = TFAutoModelForSequenceClassification.from_pretrained(
    'distilbert-base-uncased', num_labels=2
)

# Apply a low learning rate (default for Adam is 1e-4)
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=2e-5),
    loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
    metrics=['accuracy']
)

# Train for a limited number of epochs
model.fit(train_data, validation_data=validation_data, epochs=3)
```

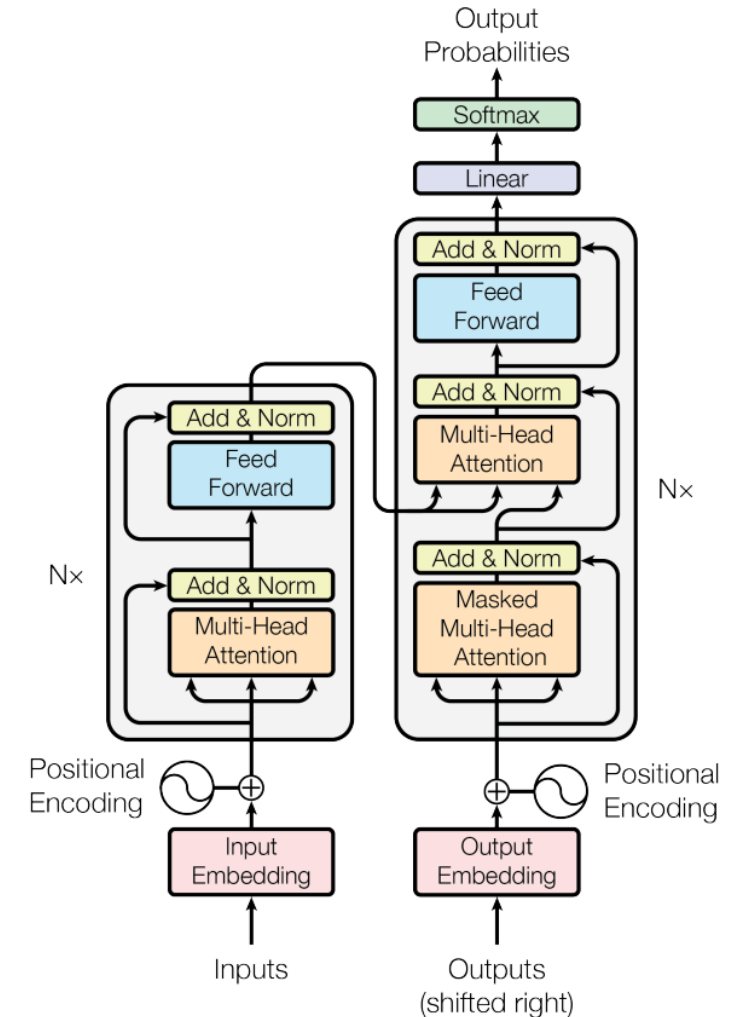
Demo

Fine-Tuning BERT for Sentiment Analysis



Encoder-Decoder Models

- Excel at neural machine translation, document summarization, and other NLP tasks
 - Input tokenized input phrase to encoder
 - Input 1) encoder output and 2) tokens predicted thus far to decoder
 - Output is softmax probabilities for all tokens in the model's vocabulary identifying the next token
- Autoregressive operation generates output by predicting one token at a time



Translating Text

hello

world

[start]

salut (65.05%)

bonjour (18.31%)

change (3.88%)

faites (0.76%)

bravo (0.73%)

Translating Text, Cont.

hello

world

[start]

salut

le (83.57%)
les (5.76%)
des (3.77%)
du (1.69%)
[end] (1.61%)

Translating Text, Cont.

hello

world

[start]

salut

le

monde (98.47%)
ton (0.13%)
credule (0.08%)
bon (0.08%)
record (0.06%)

Translating Text, Cont.

hello

world

[start]

salut

le

monde

[end] (99.18%)

est (0.53%)

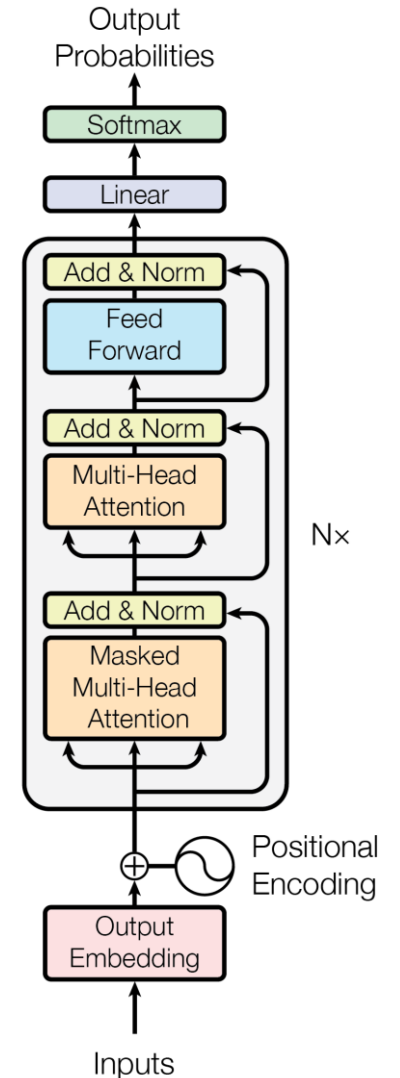
se (0.08%)

a (0.04%)

le (0.03%)

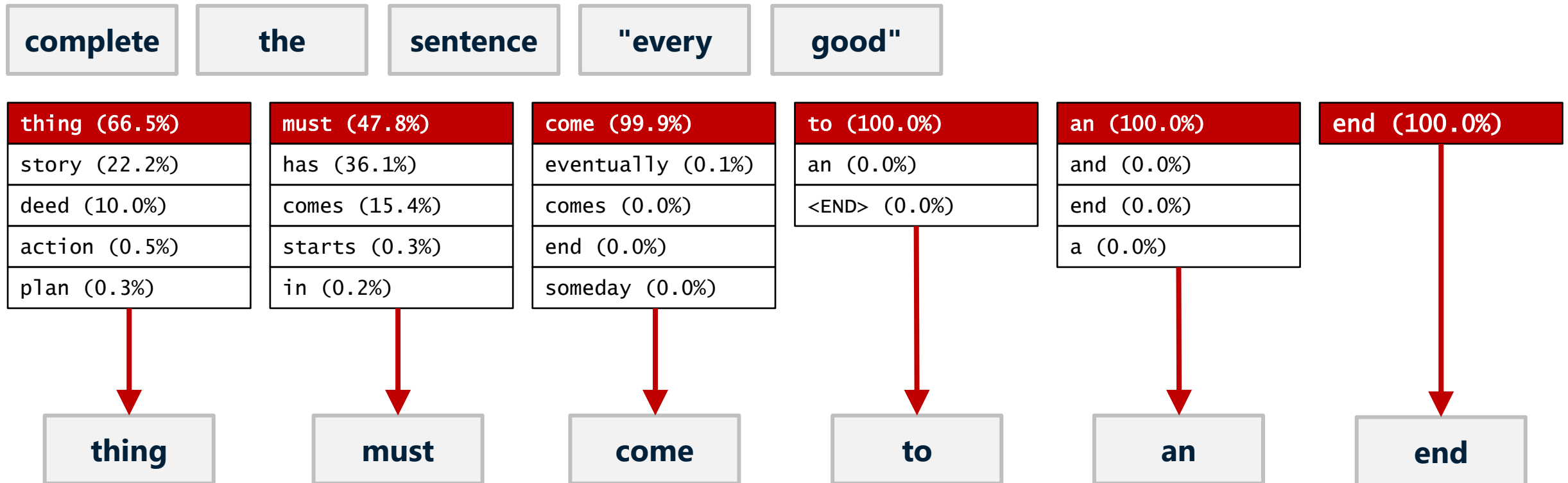
Decoder-Only Models

- Excel at text generation and serve as the basis for virtually all modern language models
 - Input is tokenized prompt + tokens generated so far
 - Output is softmax probabilities for all tokens in the model's vocabulary (typically 200K or more)
 - Sampling technique may involve more than simply picking the token with the highest probability
- Autoregressive operation generates output by predicting one token at a time



Greedy Sampling

temperature=0.0



Pure Sampling

temperature=1.0

complete

the

sentence

"every

good"

thing (66.5%)

story (22.2%)

deed (10.0%)

action (0.5%)

plan (0.3%)

must (47.8%)

has (36.1%)

comes (15.4%)

starts (0.3%)

in (0.2%)

an (67.3%)

its (31.2%)

a (0.6%)

to (0.3%)

it (0.0%)

time (38.6%)

own (21.8%)

price (20.0%)

opposite (6.3%)

season (3.0%)

thing

has

its

price

Nucleus Sampling

- Controlled by **top_p** (default=1.0)
- Lower values restrict tokens in the sampling pool by including minimum number of tokens whose cumulative probability $\geq$ **top_p**

top_p=0.8

time (38.6%)
own (21.8%)
price (20.0%)
opposite (6.3%)
season (3.0%)
fortune (1.0%)
good (0.5%)
turn (0.2%)

0.804

top_p=0.5

time (38.6%)
own (21.8%)
price (20.0%)
opposite (6.3%)
season (3.0%)
fortune (1.0%)
good (0.5%)
turn (0.2%)

0.604

top_p=0.2

time (38.6%)
own (21.8%)
price (20.0%)
opposite (6.3%)
season (3.0%)
fortune (1.0%)
good (0.5%)
turn (0.2%)

0.386

Hugging Face Transformers

- Thousands of pretrained transformer models for image classification, object detection, neural machine translation, question answering, text classification, document summarization, and more
 - Models are free and many can be used as-is (without additional training)
 - Also includes several pretrained BERT models that can be used as-is or fine-tuned to perform domain-specific tasks
- Install Python **transformers** package, which also requires TensorFlow or PyTorch

Analyzing Sentiment

```
from transformers import pipeline
```

```
model = pipeline('sentiment-analysis')
```

```
result = model('Great food and excellent service!')
```

```
# Output: [{'label': 'POSITIVE', 'score': 0.9998843669891357}]
```

Demo

Hugging Face Transformers



Summary and Conclusions

- Recurrent Neural Networks (RNN)
 - Process sequential data
 - Feed as input: hidden layer of previous time step with input of current time step
- Transformers
 - Modern network architecture using self-attention
 - Addresses problem with vanishing gradient and speed of training
- Conclusion: self-attention is the breakthrough architecture that resulted in exponential AI growth